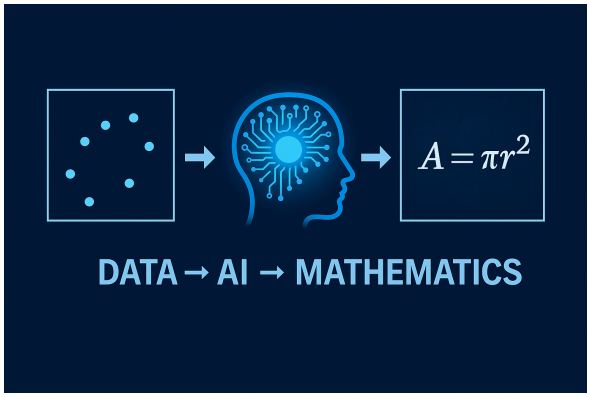
LEARNING MATHEMATICS THROUGH THE MACHINE

DR. JIMMIE ADRIAZOLA, SCHOOL OF MATHEMATICAL AND STATISTICAL SCIENCES, ASU

Big Picture Idea



At its heart, this project asks

Can machines, built on mathematics, be taught to discover mathematics itself?

To help answer this question, we adapt modern statistical learning techniques to the familiar setting of college-level calculus. Our goal is to show that even simple algorithms, running on ordinary computers, can rediscover classical mathematical formulas such as the area of a circle, the prime number theorem, or the laws of logarithms. That is, these rediscoveries are not because these rules were preprogrammed, but because the methods themselves uncover the patterns directly from data.

Now imagine that same algorithm stumbling upon a brand-new identity, a hidden symmetry, or a surprising relation that no one has ever written down before. That is the thrill we are chasing. We are after the possibility that modest, accessible tools in the hands of students can both illuminate classical mathematics and chart entirely new territory. In doing so, we not only spark discovery, but also transform the student learning experience. This makes the act of "doing math" feel less like memorization and more like participating in the grand adventure of uncovering truth.

Project Context and Impact

This proposed project aims to use a novel, albeit elementary, statistical learning and numerical optimization methodology to rediscover several famous mathematical expressions and relations, especially those arising in college-level calculus and analysis. Despite its simplicity and context within an extremely active area of research (using artificial intelligence (AI) to reason mathematically), the project has many elements that remain relatively underexplored by the research community. Indeed, the proposed principal investigator has recently demonstrated that the basic essence of the methodology, outlined later in this proposal, has made significant discoveries in more complex settings within mathematical physics.

The proposed research project retains a high degree of accessibility for online mathematics and data science undergraduate students at ASU, since many of the mathematical ideas are introduced to our students by the end of their third year in college. Algorithm execution is made accessible because the enabling computational tools, based on numerical linear algebra and optimization, can easily be handled by pre-packaged routines found in almost any modern programming language. We especially design algorithms so that they remain efficient enough to run wherever access to significant computational resources may be limited or unavailable.

The research seeks to go beyond the rediscovery of known formulae. In the best-case scenario, our methods will lead to discoveries of entirely new mathematical relationships, perhaps enabling the resolution of open mathematical problems. Indeed, we propose such problems at the end of this proposal. Of course, the risk associated with proposing student-driven research that aims to discover new and significant mathematics within a short time frame is quite high. To mitigate this risk, we attempt to demonstrate that our machine learning-driven framework has the potential to emerge as a cutting-edge pedagogical tool that can be later integrated into the ASU data science curriculum.

Through this project, our online mathematics students will benefit from further exposure to data-driven mathematics and statistical learning at ASU. It is no secret that, in addition to being an emerging and enabling methodology for mathematical reasoning, data-driven learning is a workhorse in today's scientific and technological enterprises. This research project, therefore, has the added benefit that some of our online students will be even better trained to take on the many difficult mathematical and computational problems that they will inevitably encounter during their professional careers.

We hope that you will share in our excitement for this project as you read our proposal. We first describe the expected student research experience and then move on to our plan to overcome potential barriers to student

participation. Afterwards, we present our methodology in nearly complete detail when applied to *learning the area of the circle*. We intend to have our undergraduate researchers solve this warm-up problem with clear guidance towards the intended solution. With this warm-up problem addressed, we go on to propose some more symbolic reasoning tasks that our undergraduate researchers will be equipped to tackle. We conclude the proposal with several future directions that are made possible by the successful funding and execution of this student-driven project. At the end of the document, we share the code implementation for the following warm-up problem, showing that our implementation is a neat and compact 150 lines of Matlab code, including helpful comments that explain what each block of code achieves.

Description of Research Experience

Purpose & Learning Outcomes.
Students rediscover closed-form mathematics from numerical data using the **LSSR** pipeline (Least-Squares → Sparsify → Refine), then justify their conjectures with calculus/linear algebra arguments. The LSSR pipeline is elucidated in the demonstration that **machines can rediscover the area of the circle** shown later on in this proposal. We anticipate the workflow students will learn for almost all the problems they will encounter throughout this research project will look similar to the following:

LSSR Workflow

(1) *Generate Data:* Design inputs; generate trustworthy outputs (by numerical calculus methods with tolerance control).
(2) *Fit:* Solve least squares linear algebra problems, sparsify small coefficients, refine with numerical optimization on a subspace meant for human interpretation.
(3) *Validate:* Use various standard statistical tests to validate results.
(4) *Interpret:* Convert the human interpretable model into a bona fide mathematical conjecture. Then, give a mathematical proof, thus **converting numbers from a machine into real mathematical knowledge.**
(5) *Communicate:* Make a figure that matches the claim; write a clear LaTeX paragraph; push code + data for reproduction.

To execute this research pipeline, students will integrate content across many different courses, helping them understand the bigger picture of applied and computational mathematics reserach. Here is a summary of the skills that our students encounter within the ASU applied math/data science curriculum that will be emphasized throughout this research project:

Skills Learned and Integrated in this Research Project

- *Calculus & Geometry:* The project helps reinforce how calculus is used to solve geometry problems.
- *Linear algebra & Optimization:* Computations are framed in the language of linear algebra and numerical optimization. Students will leave with an appreciation for the interplay between modern numerical linear algebra and optimization methods to solve difficult problems.
- *Scientific computing:* High-accuracy numerical calculus methods with reproducible codes and scripts.
- *Statistics & validation:* empirical risk, train/validation splits, robustness checks, and cross-validation.
- *Mathematical modeling:* Turning a statement like "$g(r) = \pi r^2$" into a data-generating process $r \mapsto g(r)$ and a hypothesis class (polynomials, orthogonal bases) is a form of modeling, not of some scientific phenomena, but mathematics itself.
- *Communication:* Students will learn to professionally communicate their research results. This includes LaTeX write-ups, clear figures (made in Inkscape/Matplotlib for example), short talks, and code documentation.
- *Collaboration & tooling:* Git/GitHub (or shared repo), issue tracking, and an always-on **Slack/Discord** channel for Q&A, peer help, and collaboration.

Structure & Schedule.

Synchronous (group, virtual). Tuesdays & Fridays.

- **Tuesday (seminar and discourse, 60–75 min):** mini-lecture (methods/papers), and student lightning updates
- **Friday (lab meeting, 60–75 min):** live code/figure reviews, experiment design, peer feedback, next-week goals.

Asynchronous.

- **Slack/Discord:** channels `#announcements`, `#help`, `#math-proofs`, `#experiments`, `#figures`. Daily check-in thread; office-hours thread for quick help.
- **Weekly cadence:**

(1) Mon–Thu: run experiments, push code, post plots and short notes.
(2) Wed: midweek progress ping on Slack/Discord.
(3) Fri: brief written update (1–2 paragraphs + 1 figure) in the repo/wiki.

Detailed 8-week timeline over the Summer

(1) *Week 1: Onboarding & Core Tools.* Git/GitHub Classroom, Google Colab or local Python/MATLAB, LaTeX and figure basics. Linear-algebra refresher: projections, least squares, normal equations.
(2) *Week 2: Numerical Integration & Conditioning.* Trapezoid/Simpson rules, error control, rescaling to $[-1, 1]$. Vandermonde vs orthogonal polynomials (Legendre/Chebyshev). QR/SVD solves; why we avoid $(D^\top D)^{-1}$.
(3) *Week 3: Sparsity, Refinement, Validation, and First Implementation.* Thresholding heuristics; quasi-Newton (BFGS) refinement on the reduced subspace. Train/validation split, error bars on coefficients. Figure style guide (Inkscape/Matplotlib). LSSR finally fully implemented for the circle area.
(4) *Week 4: Non-polynomial integrals.* Learn that $\arcsin x = \int_0^x \frac{1}{\sqrt{1-t^2}}\,dt$ (learn the series coefficients), possibly adapt to the error function erf or normal CDF Φ (fit with rational/Chebyshev bases).
(5) *Week 4: ODE Solvers & Project Setup.* Euler's Methods, Runge-Kutta, and adaptive step control toward generating trustworthy data. Help students toward building trustworthy numerical solvers for the following week. This is also a week to recap and clean up anything still lingering from the full LSSR pipeline.
(6) *Week 5: Learning Differential Equations* Scalar linear ODE $x' = ax,\ x(0) = 1$: recover e^{at}. Logistic growth $x' = rx(1 - x)$: short-time expansion; fit low-order series with LSSR. The deliverable here is a note showing recovery of the correct coefficients and robustness tests.
(7) *Weeks 6 —8: (Bonus).* If students remain on track, then students can choose any of the challenge problems listed below. Otherwise, we can use these as buffer weeks. Week 8 concludes the project and students report out, safely documenting their codes and write ups for future reference.

Potential Barriers for Student Participation. We designed this online, group-based research course to run on limited resources so students can participate regardless of bandwidth, hardware, time zone, or prior preparation. All computation is lightweight (CPU-only, minutes on a basic laptop) and uses free, widely available tools (Python/Jupyter via Google Colab or local install; GitHub Classroom templates), with no paid software required. To reduce bandwidth barriers, every Tuesday/Friday meeting is recorded with captions and shared notes; slide PDFs and code notebooks are posted in advance; low-bandwidth "audio-only + chat" participation is supported; and all tasks have asynchronous paths through Slack/Discord. To address time-zone, work, and caregiving conflicts, deadlines are flexible within weekly windows, paired with short written updates instead of long live demos. We minimize skill gaps through short primers (linear algebra, calculus refreshers, Git 101), starter notebooks, office-hours threads, and peer code review. Accessibility is considered throughout with readable fonts and color-blind-safe figures. Finally, projects are scoped to small, reproducible experiments, so no one is excluded by compute limits while still engaging in authentic, collaborative research.

Plan for Continual Progress and Sustainability

The PI will partner with Professor Marko Samara (SoMSS faculty and online data science program coordinator) to broadly advertise this proposed OURS project. After the pilot, we intend to institutionalize the project as a low-cost, credit-bearing "Research Experience" that runs each or every other year with rotating faculty leads and potentially trained peer-mentors drawn from prior cohorts. All materials—mini-lectures, notebooks, datasets, figure templates, and grading rubrics—will be packaged as reusable GitHub Classroom templates and short recorded modules so the course can scale with minimal faculty time. There is potential for the unit to maintain a persistent Slack/Discord for cross-cohort mentoring and Q&A, and nominate 2–3 returning students each term as paid (or for-credit) undergraduate research leaders who host office hours, review code, and onboard newcomers. To ensure continuity and improvement, we will keep a living problem bank (warm-ups, ODEs, non-polynomial integrals, and vetted open problems), require reproducible repos for all projects, and collect brief exit surveys to refine prompts and tooling. Sustainability costs are modest (no paid software; CPU-only workflows). These costs will be covered by a small recurring line in the unit's undergraduate research budget, supplemented by faculty course releases or a 1-credit load, and pursued longer-term through internal teaching innovation grants and external IUSE/REU proposals.

Of course, for this program to exist long term, the bank of more difficult and open problems is of high value. For this reason, we present to you, after the warm up problem, a few that we were able to put together, and one could imagine that this list, in its ambition and scope, will only grow if this program persists.

The Warm-Up Problem: Learning the Area of a Circle

This section presents the mathematical and computational details underpinning the methodology used to discover mathematics with machine learning. A simple example of a problem captured by this proposal's main research question (can machines learn math?) is the following:

Problem: Can a calculus-informed algorithm "learn" that the area of a circle is πr^2?
Answer: Yes! How? Read on...

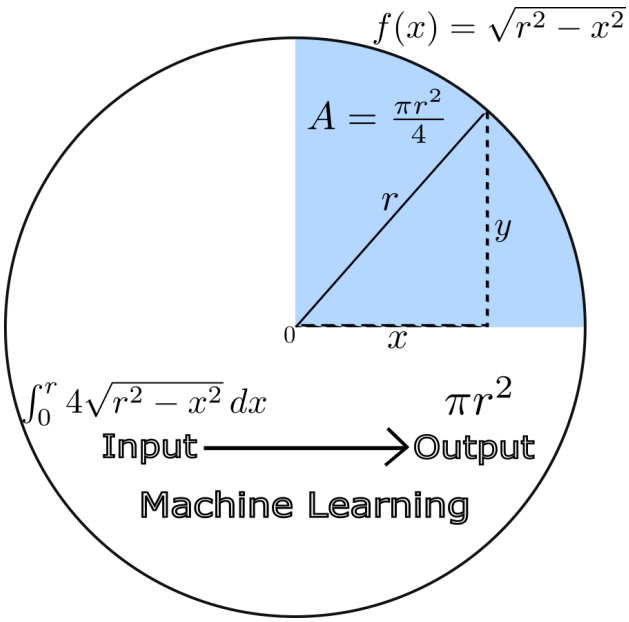
First, by calculus-informed, we mean that the algorithm is told that the area of a circle of radius r can be derived from the calculation of the following integral

$$g(r) = \int_0^r 4\sqrt{r^2 - x^2}\, dx.$$

Of course, by the end of MAT 271: Calculus with Analytic Geometry II, our students know that the function $g(r)$ is equivalent to the area of the circle because they are taught how to use polar coordinates:

$$g(r) = 4\int_0^r \sqrt{r^2 - x^2}\, dx = 4\int_0^{\pi/2} r^2 \cos^2\theta\, d\theta = \pi r^2.$$

Even though it is commonly taught, the idea to use polar coordinates is fairly nontrivial. For a generic calculation, knowledge of the right coordinate system to study a problem is not automatic. Our goal then is to circumvent the usage of a human-specified coordinate transformation and instead use an honest and good faith implementation of a **statistical learning** approach that seeks to rediscover the symbolic relation $g(r) = \pi r^2$.



Of all possible *mathematical* models of the area of a circle of radius r, $A = \pi r^2$ is the best, as it is *exactly* correct for *any* real number r. By using statistical learning, we look to build a *statistical* model that is *approximately* correct for *almost any* real number r. To enable statistical learning, we then just need two things: a statistical model and a means of training it. When a statistical model performs well on data outside of its training set, we say that the model **generalizes**. The process of quantifying how well a model generalizes is called **cross-validation**. Building a statistical model that generalizes during cross-validation is the central goal of **machine learning**.

We are left with generating training/test data and building a statistical model. In this context, data generation is easily handled by **sampling**. Sampling here means that we generate a random and finite number of points from the *space* of possible radii. To this end, we let $\{r_k\}_{k=1}^N$ denote a collection of N samples drawn uniformly from the interval $[a, b]$, that is, in statistics notation $r_k \sim \text{Uniform}(a, b)$. Since the space of all possible radii is the set of all real numbers, we see that the training data set is only a fairly limited representation of the entire data space, and the test data set is then simply data outside the interval $[a, b]$.

To build a statistical model, we must make a **hypothesis**. Let us make the simple hypothesis that the integral $g(r)$ can be modeled by the following degree-m polynomial:

$$\tilde{A}(c, r) = c_0 + c_1 r + c_2 r^2 + \ldots + c_m r^m := \sum_{j=0}^{m} c_j r^j,$$

where the $(m+1)-$dimensional column vector $c = (c_0, c_1, \ldots, c_m)^\top$ holds the unknown coefficients we seek to learn. The second ingredient to building the statistical model is to introduce a **loss function**. Using a loss function allows us to use numerical methods to perform a **regression** on the training data set. We make the natural choice to use the following quadratic loss function

$$\mathcal{L}(c; r) = \frac{1}{2}\left(\tilde{A}(c, r) - g(r)\right)^2,$$

where g evaluated at the point r is understood as a numerical approximation of the integral. That is, we may use a method such as the P-point left hand Riemann sum, introduced to our students as early as MAT 265:

Calculus for Engineers I, with equal grid spacing h to approximate the integral in the following way

$$g(r) = 4\int_0^r \sqrt{r^2 - x^2} = 4h\sum_{i=0}^{P-1}\sqrt{r^2 - x_i^2} + \mathcal{O}(h),$$

where $\mathcal{O}(h)$ denotes a global truncation error that decreases linearly as one refines the grid spacing h. The approximation by the Riemann sum is made by discarding this error term entirely.

Clearly, when the integration is exact, the best vector c^*, with regards to the loss function $\mathcal{L}$, has all components $c_j^* = 0$ except $c_2^* = \pi$. Therefore, the rediscovery of the circle's area formula is simply to find the vector c^* achieving the minimum possible value $\mathcal{L}(c^*; r) = 0$ for all real numbers r. Since we cannot access the continuous nature of all real numbers, we instead relax the problem to one in which we try to find the vector c^* such that $\mathcal{L}(c^*; r) = 0$ *on average*. This is the idea behind using an **expected value**, denoted $\mathbb{E}$, over a **probability distribution**, denoted ρ, from which we generate our training data by sampling.

With all of these pieces in hand, we formulate the machine learning of the circle's area as the following *theoretical* minimization problem:

$$\min_{c\in\mathbb{R}^{m+1}} \mathcal{J}[c] = \min_{c\in\mathbb{R}^{m+1}} \mathbb{E}_{r\sim\rho}\mathcal{L}[c; r]$$

By the law of large numbers, it should be sufficient to sample a finite, albeit many, number of times N from the distribution $\rho = \text{Uniform}(a, b)$ to build a robust statistical model $\tilde{A}(c^*, r)$ that generalizes to unseen r. Therefore, by using an **empirical mean** as an approximation of the expected value, our task now takes the form of the following *numerical* minimization problem

$$\min_{c\in\mathbb{R}^{m+1}} J[c] = \frac{1}{N}\sum_{k=1}^{N}\mathcal{L}(c; r_k).$$

In other words, the purpose of introducing an algorithm to solve this numerical optimization problem is to minimize the average squared difference between the true value of the integral and the hypothesized model. That is, this is a standard least squares problem in disguise and can be reformulated in the language of linear algebra. The main observation in this vein stems from the fact that the polynomial hypothesis, evaluated at the sampled point r_k, can be rewritten as the following inner (dot) product between two vectors

$$\tilde{A}(c, r_k) = \sum_{j=0}^{m} r_k^j c_j = R_k^\top c := \langle R_k, c\rangle,$$

where $R_k = (1, r_k, r_k^2, \ldots, r_k^m)^\top$.

To complete this picture, we define the *Vandermonde* matrix whose k-th row is $R_k^\top$, denoted by $D \in \mathbb{R}^{N\times(m+1)}$, and an auxiliary *observation* vector $b = (g_1, g_2, \ldots, g_N)^T$ where each $g_k = \int_0^{r_k} 4\sqrt{r_k^2 - x^2}\,dx$ is a numerical evaluation of the integral $g(r)$ at r_k. To be explicit, we visualize the design matrix D and the vector b by

$$D = \begin{bmatrix} 1 & r_1 & r_1^2 & \cdots & r_1^m \\ 1 & r_2 & r_2^2 & \cdots & r_2^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & r_N & r_N^2 & \cdots & r_N^m \end{bmatrix}, \qquad b = \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_N \end{bmatrix}.$$

These definitions enable us to rewrite the empirical loss function as

$$\begin{aligned}\sum_{k=1}^{N}\mathcal{L}(c; r_k) &= \sum_{k=1}^{N}\Big(\tilde{A}(c, r_k) - g(r_k)\Big)^2 \\ &= \sum_{k=1}^{N}\left(\langle R_k, c\rangle - g(r_k)\right)^2 \\ &= \sum_{k=1}^{N}(Dc - b)_k^2 := \|Dc - b\|_{l^2(\mathbb{R}^N)}^2.\end{aligned}$$

We finally arrive at the final form of our numerical optimization problem

$$\min_{c\in\mathbb{R}^{m+1}} J[c] = \min_{c\in\mathbb{R}^{m+1}} \frac{1}{2N}\|Dc - b\|_{l^2(\mathbb{R}^N)}^2.$$

This is a classic linear least squares regression that all students learn in MAT 343: Applied Linear Algebra. In that course, they also learn that if each sampled point r_j is pairwise distinct from each other, then the explicit solution, in this sense of least squares, is given by

$$c^{\text{lsq}} = \left(D^\top D\right)^{-1} D^\top b.$$

Because this research aims at mathematical reasoning, we aim for our model to be as interpretable as possible. This introduces an extra step in the numerical methodology that we must now carefully consider. We know that the polynomial hypothesis is wider than necessary, especially when m is much larger than two. And yet, as we show in Figure 1, the solution vector c^{lsq} has many, although small, nonzero components.

Therefore, we use a naive strategy to steer our numerics toward *parsimonious* model discovery. The strategy is to eliminate small components within some tolerance level. After which we perform a direct optimization to minimize the loss function in the subspace of non-zero parameters. This entire methodology is captured by Algorithm (LSSR) with all results visualized in Figure 1.

Algorithm (LSSR): Least Square–Sparsify–Refine
INPUT: Vandermonde design matrix $D \in \mathbb{R}^{N\times(m+1)}$, observation $b \in \mathbb{R}^N$, and thresholding parameter $\epsilon > 0$.
OUTPUT: The optimal and parsimonious coeffcient vector $c^\star \in \mathbb{R}^{m+1}$.

(1) **Initial fit (least-squares).** Compute $c^{\text{lsq}} = \left(D^\top D\right)^{-1} D^\top b$.
(2) **Sparsification (thresholding).** For $j = 0, 1, \ldots, m$, set $c_j^{\text{sparse}} = 0$ if $|c_j^{\text{lsq}}| < \epsilon$; otherwise $c_j^{\text{sparse}} = c_j^{\text{lsq}}$.
(3) **Refinement (subspace optimization).** Starting from c^{sparse}, minimize the empirical mean $J[c]$ using standard numerical optimization, but only in the subspace of nonzero elements; return $c^\star$.

We summarize our key findings here:

- This approach was found to be robust and reliable up to a 21 parameter hypothesis.
- We achieve our results without resorting to a more standard LASSO method, which introduces a penalization on the polynomial weights in $l^1(\mathbb{R}^{m+1})$ to promote sparsity.
- Before sparsification, the least squares solution has an error, i.e., $\|c^{\text{exact}} - c^{\text{lsq}}\|_{l^2(\mathbb{R}^{m+1})}$ that is about 0.3. Post sparsification and further optimization, the error is remarkably nine orders of magnitude smaller.
- We observe that more data does not improve generalization beyond the precision of the numerical integrator after about 30 samples. Here, we use sampled points $r_k \sim \text{Uniform}(1, 5)$ and the trapezoidal rule with enough grid points so that the numerics are accurate to single precision. This observation of training saturation is shown in the right panel of Figure 1, and justifies training on just $N = 30$ samples.

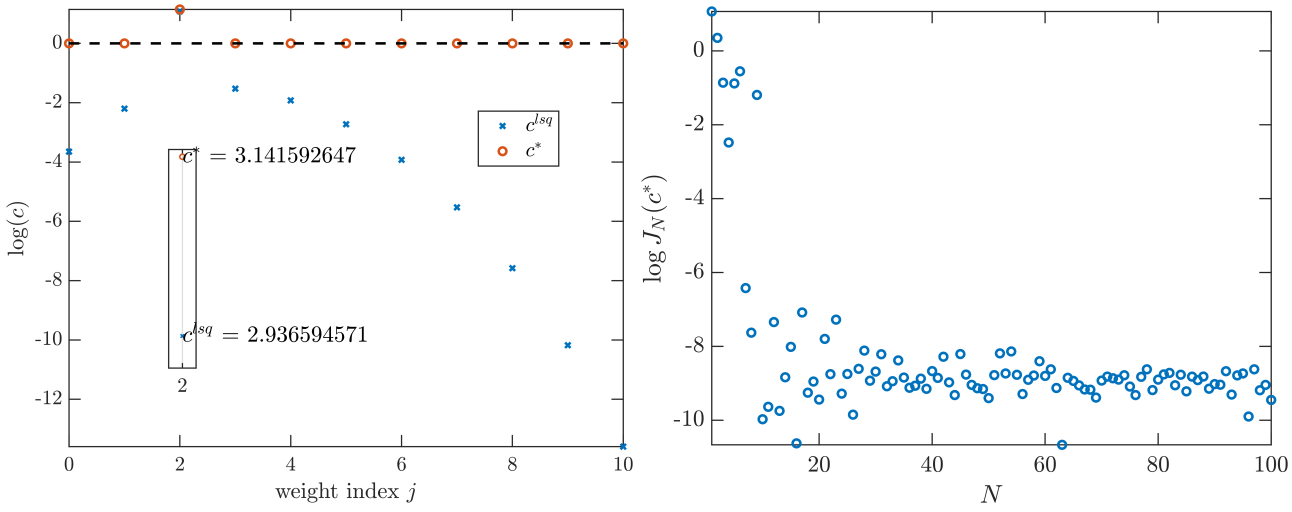


FIGURE 1. The left panel shows a comparison of the least squares solution c^{lsq} of the learning task versus the output of the LSSR Algorithm, c^*, using a 10th degree polynomial hypothesis and $N = 30$ training samples. We see the least squares solution provides a good initial guess, but after thresholding and optimizing, c^* emerges as the only expected nonzero number π, to single precision. The right panel shows the expected gain from training on larger data sets is small after about $N = 30$.

In summary, we use a data-driven approach to successfully learn a symbolic approximation of a known integral. By minimizing the empirical mean and promoting sparsity in our coefficients, we effectively recovered $g(r) = c_2^* r^2$, with $|c_2^* - \pi| < 10^{-8}$. Because of how parsimonious and accurate our statistical model is, the error of our model is quite small, only being on the order of one percent when the radius is on the order of one million!

Our methodology, in principle, is easy to adapt to other scenarios, even when the integral formula is unknown. To our knowledge, the connection of symbolic integration with standard tools of numerical analysis and supervised learning in such a didactic fashion remains relatively unexplored. Moreover, we are unaware of such basic statistical approaches, especially ones that avoid deep learning, that perform symbolic computations to

such high accuracy. Therefore, we embark on our research journey toward pushing our numerical methodology to new territory.

Proposed Research Problems

With this context in mind, our proposed project explores how known formulas can be rediscovered using statistical learning. Below are a few more examples of approachable targets that lend themselves to symbolic learning via polynomial regression. These are problems students will study after they have addressed the problem of the previous section.

Learn Solutions of Differential Equations.

As a basic model problem, consider

$$\frac{dx}{dt} = a\,x, \qquad x(0) = 1,$$

whose exact solution is $x(t;a) = e^{at} = 1 + at + \frac{(at)^2}{2!} + \frac{(at)^3}{3!} + \cdots$. Let the hypothesis be a bivariate polynomial

$$\tilde{x}(a,t) = \sum_{m=0}^{M}\sum_{n=0}^{N} A_{mn}\, a^m t^n.$$

As before, we sample parameters a from a simple distribution (e.g., uniform on $[a_{\min}, a_{\max}]$), choose a time grid $t \in [0,T]$, and obtain $x_{\text{num}}(a,t)$ (either by high-accuracy ODE solve or the exact e^{at}). We then fit A_{mn} by minimizing the expected time–integrated squared error

$$\min_A J[A] = \min_A \; \mathbb{E}_a\left[\int_0^T \big(x_{\text{num}}(a,t) - \tilde{x}(a,t)\big)^2\, dt\right].$$

Using the LSSR (least–squares → sparsify → refine) algorithm as before, we should recover the optimal coefficient matrix is diagonal with

$$A_{mn} = \begin{cases} \dfrac{1}{n!}, & m = n, \\ 0, & m \neq n, \end{cases} \qquad \text{i.e.,} \qquad \tilde{x}(a,t) = \sum_{n=0}^{\min(M,N)} \frac{a^n t^n}{n!}.$$

This essentially amounts to matching the Maclaurin series for e^z with $z = at$:

$$e^{at} = \sum_{n=0}^{\infty} \frac{(at)^n}{n!} = \sum_{n=0}^{\infty} \frac{a^n t^n}{n!},$$

and identifying coefficients of the monomials $a^m t^n$, which immediately yields $A_{mn} = \delta_{mn}/n!$.

Non-polynomial Integrals.
Here are simple integrals that yield non-polynomial functions:

$$\int_0^r \frac{1}{\sqrt{1-x^2}}\, dx = \arcsin(r),$$

and

$$\int_1^x \frac{1}{t}\, dt = \ln(x).$$

Just like with learning solutions to differential equations, we may still attempt a low-degree polynomial hypothesis, and attempt to learn the first few coefficients accurately. Then, by mathematical induction, we can prove that the algorithm assists in discovering the above non-polynomial functions.

More Difficult Integrals.

Dirichlet kernel energy:
With $D_n(\theta) = \sum_{k=-n}^{n} e^{ik\theta} = \frac{\sin\left((n+\frac{1}{2})\theta\right)}{\sin(\theta/2)}$,

$$\frac{1}{2\pi}\int_{-\pi}^{\pi} |D_n(\theta)|^2\, d\theta = 2n+1.$$

Family of semicircle-type integrals:
For $\alpha > -1$,

$$I_\alpha(r) = \int_0^r (r^2 - x^2)^\alpha\, dx = C(\alpha)\, r^{2\alpha+1}, \qquad C(\alpha) = \frac{\sqrt{\pi}\,\Gamma(\alpha+1)}{2\,\Gamma\left(\alpha+\frac{3}{2}\right)}.$$

Wallis-type integrals:

For $n \geq 0$,

$$W_n = \int_0^{\pi/2} \sin^n x \, dx = \frac{\sqrt{\pi}\,\Gamma\left(\frac{n+1}{2}\right)}{2\,\Gamma\left(\frac{n}{2}+1\right)}.$$

Parameterized moments and Beta function:
For $p, q > -1$,

$$B(p+1, q+1) = \int_0^1 x^p (1-x)^q \, dx = \frac{\Gamma(p+1)\Gamma(q+1)}{\Gamma(p+q+2)}.$$

Learning Asymptotic Relations.

Stirling's approximation:

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

Prime Number Theorem:

$$\pi(N) \sim \frac{N}{\log N}, \quad \text{where } \pi(N) \text{ is the prime-counting function.}$$

Elliptic-Integral leading asymptotics: As $m \to 1^-$,

$$K(m) = \int_0^{\pi/2} \frac{d\theta}{\sqrt{1 - m\sin^2\theta}} \sim \tfrac{1}{2}\log\left(\frac{16}{1-m}\right).$$

Well-Known Inequalities of Analysis.

Wirtinger / periodic Poincaré (mean zero). For $f \in H^1_{\text{per}}(0, 2\pi)$ with $\int_0^{2\pi} f = 0$,

$$\int_0^{2\pi} f(x)^2 \, dx \;\leq\; \int_0^{2\pi} f'(x)^2 \, dx, \quad \text{and 1 is sharp (equality for } f = \sin x).$$

On $(0, L)$ the constant is $(L/2\pi)^2$ and is sharp.

Hardy inequality on $(0, 1)$. For $f \in H^1_0(0, 1)$,

$$\int_0^1 \frac{|f(x)|^2}{x^2} \, dx \;\leq\; 4\int_0^1 |f'(x)|^2 \, dx, \quad \text{and 4 is sharp.}$$

Neumann Poincaré (mean-zero form). For $f \in H^1(0, 1)$ with $\int_0^1 f = 0$,

$$\int_0^1 f(x)^2 \, dx \;\leq\; \frac{1}{\pi^2}\int_0^1 |f'(x)|^2 \, dx, \quad \text{and } \frac{1}{\pi^2} \text{ is sharp.}$$

Bernstein inequality (trigonometric polynomials). If $T_n(\theta) = \sum_{k=-n}^{n} c_k e^{ik\theta}$,

$$\|T_n'\|_{L^\infty[0,2\pi]} \;\leq\; n\,\|T_n\|_{L^\infty[0,2\pi]}, \quad \text{and } n \text{ is sharp.}$$

Jordan's sine bounds. For $x \in (0, \frac{\pi}{2})$,

$$\frac{2}{\pi}\,x \;\leq\; \sin x \;\leq\; x, \quad \text{both constants are sharp.}$$

Isoperimetric inequality in the plane: For a planar domain with area A and perimeter P,

$$4\pi A \;\leq\; P^2, \quad \text{and equality holds iff the domain is a disk.}$$

Hölder's inequality: For $1 < p, q < \infty$ with $\frac{1}{p} + \frac{1}{q} = 1$,

$$\int |fg| \;\leq\; \|f\|_{L^p}\|g\|_{L^q}, \quad \text{constant 1 is sharp (equality when } |f|^p \propto |g|^q).$$

Baby Young's inequality: For $a, b \geq 0$ and $1 < p, q < \infty$ with $\frac{1}{p} + \frac{1}{q} = 1$,

$$ab \;\leq\; \frac{a^p}{p} + \frac{b^q}{q}, \quad \text{constant 1 is sharp (equality when } a^p = b^q).$$

Learning Sequences and Recurrence Relations.

Discover the Legendre Polynomials:
Given a weight w on $[-1,1]$, perform the Gram–Schmidt procedure on $\{1, x, \dots, x^m\}$ to construct an orthogonal family $\{p_k\}$. Then estimate moments $\mu_j = \int x^j w(x)\,dx$, and fit the three-term recurrence

$$p_{k+1}(x) = (x - a_k)p_k(x) - b_k p_{k-1}(x),$$

to discover

$$a_k = 0, \qquad b_k = \frac{k^2}{4k^2 - 1}.$$

Wallis integrals and the recurrence:
For $n = 0, 1, 2, \dots$, compute

$$W_n = \int_0^{\pi/2} \sin^n x\,dx$$

and learn

$$W_n = \frac{n-1}{n}\,W_{n-2}, \qquad W_n = \frac{\sqrt{\pi}\,\Gamma\left(\frac{n+1}{2}\right)}{2\,\Gamma\left(\frac{n}{2}+1\right)}.$$

Chebyshev–Legendre moments:
Compute

$$\mu_k = \int_{-1}^{1} x^k\,dx, \quad \ell_n = \int_{-1}^{1} P_n(x)\,dx, \quad \tau_n = \int_{-1}^{1} T_n(x)\,dx.$$

and learn

$$\mu_{2k} = \frac{2}{2k+1},\ \mu_{2k+1} = 0; \quad \ell_0 = 2,\ \ell_n = 0\ (n \geq 1); \quad \tau_{2m} = \frac{2}{1-4m^2},\ \tau_{2m+1} = 0.$$

Semicircle moments and Catalan numbers:
For $\rho_R(x) = \dfrac{2}{\pi R^2}\sqrt{R^2 - x^2}$ on $[-R, R]$, compute

$$m_{2k} = \int_{-R}^{R} x^{2k} \rho_R(x)\,dx.$$

and discover

$$m_{2k} = C_k\,\frac{R^{2k}}{2^{2k}}, \qquad C_k = \frac{1}{k+1}\binom{2k}{k}.$$

Foundational Formulas of Elementary Numerical Analysis.

Composite quadrature error constants:
For smooth f on $[a, b]$, measure the error of composite Simpson and trapezoid rules as n varies; fit

$$E_{\mathrm{S}}(n) \approx \frac{K}{n^4}, \qquad E_{\mathrm{T}}(n) \approx \frac{K'}{n^2}.$$

and discover

$$K = -\frac{(b-a)^5}{180}\,f^{(4)}(\xi), \qquad K' = -\frac{(b-a)^3}{12}\,f''(\xi)$$

for some $\xi \in (a, b)$.
Finite-difference truncation constants:
For

$$\delta_h^2 f(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2},$$

measure $f''(x) - \delta_h^2 f(x)$ as $h \to 0$ and discover

$$f''(x) - \delta_h^2 f(x) \approx \frac{1}{12}\,h^2 f^{(4)}(x).$$

Taylor remainder best constants on $[-1, 1]$
Fit the smallest C with

$$\left|\sin x - x + \tfrac{x^3}{6}\right| \leq C\,|x|^5, \qquad x \in [-1, 1].$$

and discover $C = \frac{1}{120}$, with possible interval-improved constants via maximization.

Two Major Open Problems

(1) **Erdős–Straus conjecture (Egyptian fractions)**
- **Statement:** For every $n \geq 2$, there exist $x, y, z \in \mathbb{N}$ with
$$\frac{4}{n} = \frac{1}{x} + \frac{1}{y} + \frac{1}{z}.$$
- **Lofty LSSR plan:** For each residue class of n modulo small m, *learn* sparse parametric formulas for $(x(n), y(n), z(n))$ that cover infinite families; validate at scale.

(2) **Pair Correlation Conjecture (zeros of ζ)**
- **Statement:** The pair correlation of nontrivial zeros of $\zeta(s)$ matches the Gaussian Unitary Ensemble sine-kernel law:
$$R_2(u) \stackrel{?}{=} 1 - \left(\frac{\sin \pi u}{\pi u}\right)^2.$$
- **Lofty LSSR plan:** Using zero datasets at finite height, estimate $R_2(u)$ and *fit* sparse kernels to quantify finite-size deviations. We may further conjecture correction terms and scaling windows.

Matlab Code Used to Learn the Area of the Circle

```
%% Main Script

% number of parameters
numP=11;

% number of training samples
numTrials=30;

% number of quadrature points
N=1e5;

% radial domain as a function of r
x=@(r)linspace(0,r,ceil(r*N));

% Pythagoras
f=@(r)4*sqrt(r^2-x(r).^2);

% rand samples in the interval [1,5]
rvec=4*rand(numTrials,1)+1;

% optimization options
options=optimoptions('fminunc','MaxFunctionEvaluations',1e16,...
    'OptimalityTolerance',1e-16,"StepTolerance",1e-100,"MaxIterations",1e16,"Display","none");

% build the Vandermonde matrix
rvec=sort(rvec);
for i=1:numTrials
    g(i,1)=trapz(x(rvec(i)),f(rvec(i)));
    for j=1:numP
        R(i,j)=rvec(i)^(j-1);
    end
end

% projected least squares solution
a=(R'*R)\(R'*g);

% thresholded solution
[c,thresh,filterind]=sparsify(f,rvec,a,x,numP,options);

% this is the error
cexact=0*c;cexact(3)=pi;
Error=norm(cexact-c,2);
fprintf('The observed error is %.16f. Also, observe that learned c is sparse.\n',Error)

plotArea(a,c,j)
```

```
function J=getObjective(f,rvec,a,x,filterind,DD)

% let's set up the parameter space

if ~isempty(filterind)
    % this means we're doing optimization in the subspace
    y=zeros(1,DD);
    y(filterind)=a;
    a=y;
end

% start the evaluation of the loss over samples \vec{r}
J=0;
for i=1:length(rvec)
    % this sample r
    r=rvec(i);

```

```
    % our hypothesis is a polynomial
    Hyp=0;
    for k=1:length(a)
        Hyp=Hyp+a(k)*r^(k-1);
    end

    % the loss measures how far the polynomial is from numerical integral
    J=J+.5*(Hyp-trapz(x(r),f(r)))^2;
end

% empirical mean
J=J/length(rvec);

end
```

```
function [a,thresh,filterind]=sparsify(f,rvec,x0,x,DD,options)

% threshold level is 10 percent of the max parameter value
A=max(abs(x0));
thresh=A/10;

% find me what I zeroed out
ind=find(abs(x0)<thresh);

if ~isempty(ind)
    % then zero it out
    x0(ind)=0;
end

% find me places in the original parameter space that were nonzero
filterind=find(x0~=0);
indd=find(x0==0);

% get rid of them so that the optimization takes place in the subspace
x0(indd)=[];

% define the objective and solve the optimization problem
J=@(a)getObjective(f,rvec,a,x,filterind,DD);
a=fminunc(J,x0,options);

% embed the subspace back into the original parameter space
y=zeros(DD,1);
y(filterind)=a;
a=y;

end
```

```
function plotArea(a,c,j)

% big plot in log scale
CC = c;
CC(3) = log(CC(3));
plot(0:j-1, log(abs(a)), 'x'); hold on
plot(0:j-1, CC, 'o')
plot(0:j-1, 0*c, 'k--'); axis tight
legend('$c^{lsq}$','$c^*$','FontSize', 18)
xlabel('weight index $j$','FontSize', 18)
ylabel('$\log(c)$','FontSize', 18)

%bottom-left inset showing c_2 in decimal
axMain = gca;
pos    = axMain.Position;                % [x y w h] in normalized figure units

% Even margins & size (relative to main axes)
pad = 0.18;                               % nward padding
```

```
w   = 0.05*pos(3);                          % inset width
h   = 0.50*pos(4);                          % inset height
insetPos = [pos(1)+pad*pos(3), pos(2)+pad*pos(4), w, h];

axInset = axes('Position', insetPos);
hold(axInset, 'on'); box(axInset, 'on'); grid(axInset, 'on');
set(axInset, 'FontSize', 12)

plot(axInset, 2,a(3), 'x', 'LineWidth', 1.2, 'MarkerSize', 4), hold on
plot(axInset, 2, c(3), 'o', 'LineWidth', 1.2, 'MarkerSize', 4)
xlim(axInset, [2-0.2, 2+0.2])


% Clean ticks/labels for an even look
xticks(axInset, 2); xticklabels(axInset, {'2'})
yticks(axInset, [])

% Optional numeric label
text(axInset, 2, c(3), sprintf('    $ c^*$ = %.10g', c(3)), ...
     'HorizontalAlignment','left', 'VerticalAlignment','middle', 'FontSize', 18);
text(axInset, 2, a(3), sprintf('    $  c^{lsq} $ = %.10g', a(3)), ...
     'HorizontalAlignment','left', 'VerticalAlignment','middle', 'FontSize', 18);
end
```